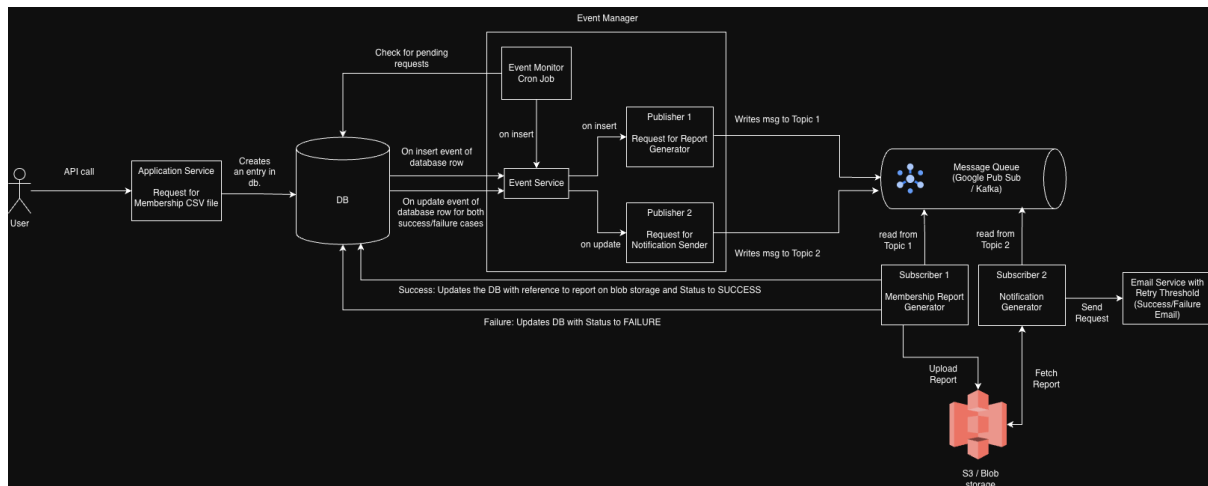


# Task 2: Architecture Diagram



## User

The end-customer who initiates a request to export their membership data as a CSV file.

## Application Service

A backend API service that exposes an endpoint allowing users to request a membership report.

Upon receiving the request, it creates a Report Job entry in the database containing:

- jobId
- userId
- status (e.g., INITIATED, SUCCESS, FAILURE)
- blobId (reference to generated CSV file in object storage, populated later)
- timestamps

## Database (with Change Data Events)

A persistent data store that holds the lifecycle state of each report generation request.

It emits two types of change events:

- **OnInsert Event:**
  - Triggered when a new report job row is created.
  - Calls Event Service to initiate report generation.
- **OnUpdate Event:**
  - Triggered when a job record is updated with either:
    - SUCCESS + blobId (report generated successfully), or
    - FAILURE (report generation failed)
  - Calls Event Service to initiate notification sending.

These change events are implemented via DB triggers.

## **Event Service**

A backend API service that are called by DB triggers.

It determines the type of event and publishes appropriate events:

- Insert event → Publishes a "Report Generation Request" message to Topic 1 via Publisher 1
- Update event → Publishes a "Notification Request" message to Topic 2 via Publisher 2

## **Event Monitor**

A cron job that tracks the status of report request jobs in DB.

Its responsibilities:

- If a job remains in the 'INITIATED' state beyond a defined threshold, it re-triggers the onInsert event on the event service.

## **Publisher 1 (Report Generation Publisher)**

A component within the Event Service responsible for serializing and publishing Report Generation Request messages to Topic 1 of the message queue.

## **Publisher 2 (Notification Publisher)**

A similar component that serializes and publishes Notification Request messages to Topic 2, triggered whenever the report job status changes to SUCCESS or FAILURE.

## **Message Queue (Kafka / Google PubSub)**

A distributed, highly-scalable message broker.

It defines:

- Topic 1: Report Generation Events
- Topic 2: Notification Events

## **Subscriber 1 (Membership Report Generator)**

A consumer subscribed to Topic 1.

Its responsibilities:

- Consume report generation messages
- Extracts membership data from messages
- Generate the CSV report
- Upload the file to S3 / Blob Storage
- Update the job record in the database:
  - SUCCESS + blobId, or
  - FAILURE

## Subscriber 2 (Notification Generator)

A consumer subscribed to Topic 2.

It processes all notification events, which include both successful and failed report generation scenarios.

Responsibilities:

- Fetch the CSV file from S3 (if successful)
- Construct the appropriate success/failure email content
- Call the Email Service to dispatch the email

## Email Service

A dedicated service responsible for sending transactional emails.

Features include:

- Support for both success and failure notification templates
- Email queueing and **retry logic with definite threshold**
- Logging of persistent failures for operational follow-up

This service is invoked by the Notification Generator.

## S3 / Blob Storage

Blob storage for storing generated CSV reports.

Characteristics:

- Holds the generated report file referenced by blobId
- Auto-deletion / TTL applied to avoid excessive storage growth